

Phase 3: A Substrate-Level Reformulation at the P vs NP Axis via the Principia Fractalis Framework

(Not a Clay Discharge; Machine Type Carries No Operational Turing Semantics)

Pablo Cohen

psolorzano@gmail.com

Collaboration: Claude Opus 4.7 (Anthropic)

2026-06-06

Abstract

This is the Phase 3 paper of a four-paper sequence on the Principia Fractalis (PF) framework’s substrate-level reformulation of the six remaining Clay Millennium Problems. **This is not a Clay-prize claim for P vs NP.** The framework’s content on this axis is: (i) the chosen value $\alpha_{\text{PvsNP}} := 5/4$ pinned by the eleven framework-internal arithmetic compatibility identities on the chosen α -skeleton under the editorial convention $\alpha_{\text{Poincaré}} := 1$ motivated by Perelman 2003; (ii) an unconditional biconditional `Clay_PvsNP_Standard PF_CanonicalComplexityEncoding` $\leftrightarrow$ (`ClassP` $\neq$ `ClassNP`) on the framework’s chosen complexity-encoding; (iii) the open Clay conjecture itself (`ClassP` $\neq$ `ClassNP` on the chosen encoding) as the named open residual the framework does *not* discharge.

The framework’s `Machine` type is *not* an operational Turing machine. Its two fields (`accepts : BinString  $\rightarrow$  Prop` and `steps : BinString  $\rightarrow$   $\mathbb{N}$`) are free functions; the three type parameters Γ, Δ, σ are unused in the body. The file `docstring` acknowledges this; the classes `ClassP` and `ClassNP` built atop this structure are substrate-shadows of the complexity-theoretic structure, not the operational Cook–Karp form. The framework’s contribution on this axis is the biconditional and the α -skeleton pinning; it is *not* a constructive polynomial-time algorithm for SAT, and it is *not* a literal mathlib-tier discharge of $\text{P} \neq \text{NP}$.

Machine-verified in Lean 4 with kernel-only axioms [`propext`, `Classical.choice`, `Quot.sound`]. The `PF_Lean4Lean` harness provides build-hash robustness across two Lake packages (*not* an independent type-checker in another language). Honest scope is preserved throughout. Independent expert review and substantial revision are explicitly invited.

Contents

1	What this paper offers and what it does not	2
2	The framework’s <code>Machine</code> type, foregrounded	2
3	The substrate and the editorial convention	3
4	The α -values on the complexity axis	3
5	The canonical complexity encoding and the biconditional	4
5.1	The framework’s chosen complexity encoding	4
5.2	The unconditional biconditional	4
6	The polylog typed upgrade — conditional content	5
7	Connection to the simultaneous-closure mechanism	5

8	Independent verification posture	5
9	What this does and does not open	6
10	Honest scope	6
11	Conclusion	7

1 What this paper offers and what it does not

This paper offers a *substrate-level reformulation of the P vs NP axis* of the Principia Fractalis framework. It does *not* offer a literal mathlib-tier discharge of $P \neq NP$ in the Clay form, and it does *not* offer a constructive polynomial-time algorithm for SAT or for any other NP-complete problem. The substrate-level content is the unconditional biconditional on the framework’s chosen encoding and the α -skeleton pinning of $\alpha_{PvsNP} := 5/4$, $\alpha_P := \sqrt{2}$, $\alpha_{NP} := \varphi + 1/4$; the open Clay conjecture $ClassP \neq ClassNP$ on the chosen encoding is preserved openly.

What the Lean kernel carries on this axis. The biconditional `Clay_PvsNP_Standard PF.CanonicalComplexityEncoding` $\leftrightarrow$ $(ClassP \neq ClassNP)$ unconditional on the framework’s chosen encoding; the chosen $\alpha_{PvsNP} := 5/4$ value as the unique solution to the framework’s algebraic system under the editorial convention; the bundle field `pnp_classes_distinct : ClassP  $\neq$  ClassNP` from the simultaneous-closure bundle of Phase 2 (this field *is* the P vs NP question on the chosen encoding, preserved openly).

What this paper does NOT claim. A literal mathlib-tier discharge of $P \neq NP$ (the form required by the Clay rules). A constructive polynomial-time algorithm for any NP-complete problem. A separation through a small-polynomial separator. The framework’s `Machine` type carries no operational Turing semantics; on this encoding, the discharge is a substrate-shadow of the Cook–Karp operational form, not the operational form itself.

2 The framework’s Machine type, foregrounded

The framework’s complexity-axis encoding lives in `PrincipiaTractalis.TuringEncoding.Complexity`. Its `Machine` structure is defined (line 71) as:

```
structure Machine (Gamma Lambda sigma : Type) where
  accepts : BinString -> Prop
  steps   : BinString -> N
```

Honest scope, foregrounded:

- The three type parameters Γ, Λ, σ are *unused* in the body of the structure.
- The two fields `accepts` and `steps` are *free functions* with no operational semantics constraining them to be Turing-computable.

The file `docstring` at the structure (lines 57–70 of `Complexity.lean`) acknowledges this: “*the full mathlib Turing.TM2 model parametrizes over K (stack indices), $\Gamma : K \rightarrow \text{Type}$ (alphabet per stack), Λ (state set), and σ (auxiliary register). Here we keep the surface signature...*”

The classes `ClassP` (line 118) and `ClassNP` (line 158) built atop this structure are therefore *substrate-shadows* of the Cook 1971 / Karp 1972 operational form, not the operational form itself. The framework’s `InClassP` predicate says “there exists a machine M with `steps` polynomially bounded and `accepts` agreeing with membership in L ”; but the existential ranges over the free-function `Machine` population, not over operational Turing machines, so its meaning is the framework’s substrate-shadow of the operational class.

What this means for the framework’s P vs NP claim. The biconditional and the $ClassP \neq ClassNP$ formulation are correctly stated *on the framework’s chosen encoding*. They

are not the literal Cook–Karp operational formulation, and the framework does not claim they are. A literal mathlib-tier discharge would require either porting the framework’s encoding to mathlib’s `Turing.TM2` or constructing a specific machine inhabiting the predicates; neither is done here. The open Clay conjecture, in the framework’s chosen encoding, is the bundle field `pnpclasses_distinct` of Phase 2’s seven-field bundle.

3 The substrate and the editorial convention

The framework’s substrate is the Timeless Field $\mathcal{H}_{\text{TF}} = \varprojlim_k \mathbb{C}^{3^k}$ (see the master paper [1] for the substrate description). The editorial convention $\alpha_{\text{Poincaré}} := 1$ is motivated by Perelman 2003’s resolution of the Poincaré conjecture [10, 11, 12]; the Lean kernel does not consume Perelman’s theorem as a hypothesis. The convention is the framework’s identification of the substrate’s Poincaré-axis constant.

The eleven framework-internal arithmetic compatibility identities on the chosen α -skeleton are catalogued in the master paper and in Phase 2. Honest scope, quoted verbatim from `CrossMillenniumSharedInvariants.lean` (lines 18–29): “*These are **not** Millennium discharges. They are axiom-free algebraic facts about the 9 chosen α -values; the framework’s interpretation... is conjectural and lives elsewhere.*”

4 The α -values on the complexity axis

The framework’s α -skeleton has three distinguished values on the complexity axis: $\alpha_P := \sqrt{2}$, $\alpha_{NP} := \varphi + 1/4$ (where $\varphi = (1 + \sqrt{5})/2$), and $\alpha_{\text{PvsNP}} := 5/4$, each defined in `PF.CrossMillenniumSharedInvariants`.

Theorem 4.1 (Chosen complexity-axis α -values pinned by the framework-internal identities under the editorial convention). *The chosen complexity-axis values*

$$\alpha_P = \sqrt{2}, \quad \alpha_{NP} = \varphi + 1/4, \quad \alpha_{\text{PvsNP}} = 5/4$$

are the unique solution to the framework’s algebraic system under the editorial convention $\alpha_{\text{Poincaré}} := 1$ and the eleven framework-internal arithmetic identities (e.g. $\alpha_P^2 = \alpha_{\text{YM}}$ where $\alpha_{\text{YM}} = \alpha_{\text{Poincaré}} + 1 = 2$; $\alpha_{NP} - \alpha_{\text{Hodge}} = 1/4$ where $\alpha_{\text{Hodge}} = \varphi$). **Lean:** `PF.Referee.ClayMasterTheorem.framework_alpha_unique_under_perelman_anchor; PrincipiaTractalis.CrossMillenniumSharedInvariants.cross_millennium_shared_invariants_capstone; α_P .sq.eq. α_{YM} ; α_{NP} .sub.Hodge.eq.quarter; α_{Hodge} .sq.eq.self_plus_one; α_{YM} .eq. $\alpha_{\text{Poincaré}}$.plus_one.`

Honest scope. These are arithmetic facts about *chosen* constants under the editorial convention. The framework’s interpretation that α_P and α_{NP} are the canonical eigenvalue parameters of the P and NP operators in the framework’s polylog encoding is, per `CrossMillenniumSharedInvariants.lean` header, “conjectural and lives elsewhere.”

5 The canonical complexity encoding and the biconditional

5.1 The framework’s chosen complexity encoding

The framework’s referee module `StandardClayStatements` parameterises the Clay P vs NP statement over an external encoding `StandardComplexityEncoding` (three fields: `ClassP : Type`, `ClassNP : Type`, `inclusion : ClassP → ClassNP`). The Clay-statement contract on an encoding E is

$$\text{Clay_PvsNP_Standard } E := \neg \text{Function.Surjective } E.\text{inclusion}.$$

The framework’s chosen instantiation is:

Definition 5.1 (The framework’s chosen complexity-encoding).

$$\begin{aligned}\text{CanonicalClassP} &:= \{L : \text{Language} \mid L \in \text{ClassP}\}, \\ \text{CanonicalClassNP} &:= \{L : \text{Language} \mid L \in \text{ClassNP}\},\end{aligned}$$

$\text{PF_CanonicalComplexityEncoding} := \langle \text{CanonicalClassP}, \text{CanonicalClassNP}, \text{canonical_PvsNP_inclusion} \rangle$

The inclusion $\text{canonical_PvsNP_inclusion} : \text{CanonicalClassP} \rightarrow \text{CanonicalClassNP}$ is defined by $L \mapsto \langle L.\text{val}, \text{P_subset_NP } L.\text{property} \rangle$, where P_subset_NP is the framework’s typed proof that the framework’s ClassP is included in ClassNP . **Lean:** $\text{PF.Referee.PNPCanonicalEncoding}$. $\text{PF_CanonicalComplexityEncoding}$ (line 81).

Honest scope, foregrounded again. This encoding is built from the framework’s $\text{ClassP}/\text{ClassNP}$, which are in turn built from the framework’s Machine type whose fields are free functions with no operational Turing semantics (§2). The encoding is *the framework’s substrate-shadow* of the Cook–Karp form. Earlier framework documents that called it “the literal mathlib-canonical Cook 1971 / Karp 1972 encoding” overstated the encoding’s status; it is the framework’s chosen encoding built atop a Machine structure that does not carry operational Turing semantics. The framework’s commitment to the Cook–Karp form is structural (subtype quotient of languages satisfying the predicates) but is not validated through an operational-machine layer.

5.2 The unconditional biconditional

Theorem 5.2 (Biconditional on the chosen encoding). *On the framework’s chosen complexity-encoding,*

$$\text{Clay_PvsNP_Standard } \text{PF_CanonicalComplexityEncoding} \iff \text{ClassP} \neq \text{ClassNP}.$$

Lean: $\text{PF.Referee.PNPCanonicalEncoding}$.

Clay_PvsNP_Standard_at_canonical_iff_classes_distinct (line 92). Kernel axioms: $[\text{prope} \text{xt}, \text{Classical.choice}, \text{Quot.sound}]$.

The biconditional itself is unconditional on the chosen encoding. **The substrate-level content is the biconditional; the right-hand side ($\text{ClassP} \neq \text{ClassNP}$ on the chosen encoding) is the open Clay conjecture, preserved openly,** and appears as the bundle field $\text{pnp_classes_distinct}$ in the Phase 2 simultaneous-closure bundle.

6 The polylog typed upgrade — conditional content

The framework’s $\text{PolylogEigenvalueConjecture}$ residual ($\text{PF.TuringEncoding.Operators.PolylogEigenvalue}$) is the existential statement that there exists a class function $f : \text{Set Language} \rightarrow \mathbb{R}$ satisfying the framework’s four typed sub-Props (P-class self-adjointness, P-class positivity, NP-class golden-modulation quadratic, NP-class positivity). The framework’s typed biconditional reduces this existential to $\text{ClassP} \neq \text{ClassNP}$:

Theorem 6.1 (Polylog typed upgrade: existential $\Leftrightarrow$ class distinctness).

$$(\exists f, A1(f) \wedge A2(f) \wedge B1(f) \wedge B2(f)) \iff \text{ClassP} \neq \text{ClassNP}.$$

Lean: $\text{PrincipiaTractalis.TuringEncoding}$.

polylog_subprop_quadruple_iff_classes_distinct.

Honest scope. The biconditional is unconditional; its content is logical equivalence of the existential and class distinctness on the framework’s chosen encoding, *not* a proof of either. The four sub-Props are discharged axiom-free *at the substrate enum-level* (the framework’s enumeration on the chosen $(\alpha_P, \alpha_{NP}) = (\sqrt{2}, \varphi + 1/4)$), per $\text{polylog_subprop_quadruple_at_enum}$; this is a substrate-level discharge, not a literal mathlib-tier discharge.

7 Connection to the simultaneous-closure mechanism

The substrate-level Clay-Standard contract on the chosen P vs NP encoding is one of six axes in the Phase 2 simultaneous-closure mechanism. The bundle field `pnp_classes_distinct : ClassP  $\neq$  ClassNP` is the typed residual consumed for this axis; the simultaneous-closure theorem `perelman_anchor_yields_simultaneous_clay_closure` delivers the six Clay-Standards on canonical encodings under the editorial convention plus the seven-field bundle (5 typed residuals + 2 True-markers).

The framework’s contribution at this axis is structural: the biconditional and the α -skeleton pinning on the chosen encoding; the open Clay conjecture itself is preserved openly. See the master paper [1] for the simultaneous-closure mechanism’s full statement and per-axis honest-scope discussion.

8 Independent verification posture

The Lean 4 proof object for the theorems cited type-checks in the Lean 4 kernel with kernel axioms [`propext`, `Classical.choice`, `Quot.sound`] and zero project axioms. The `PF_Lean4Lean` harness provides **build-hash robustness across two Lake packages**: the canonical theorems are rebound in a separate package and rebuilt from source. **It is not an independent type-checker written in another language.** Quoted verbatim from `PF_Lean4Lean/PF_L4L/Referee/ClayVerificationHar` lines 47–56:

“This is the same Path-C protocol... It is NOT a separate type-checker written in another language. The honest scope of each individual closure is the one fixed by that closure’s own `*_honestScope` / `*_honest_scope` marker in the canonical PF library... The harness re-exports the closures as they are — it does NOT strengthen any of them.”

Earlier drafts called this an “independent kernel re-verifier”; that framing overstated the harness’s content and is dropped in favor of the file’s own self-description.

9 What this does and does not open

The framework’s contribution on the P vs NP axis is structural at the α -skeleton level on the chosen encoding. **The framework does not supply:**

- A constructive polynomial-time algorithm for SAT or for any other NP-complete problem.
- A literal mathlib-tier discharge of $P \neq NP$.
- A small-polynomial separator.
- A bypass of Razborov–Rudich 1997 natural-proofs [8] or Aaronson–Wigderson 2009 algebraisation [9] barriers.

What the framework supplies is the substrate-level reformulation: the α -skeleton pinning of $\alpha_{P \text{ vs } NP} = 5/4$ on the chosen encoding, the biconditional reducing the Clay-Standard contract to the open class-distinctness conjecture on the chosen encoding, and the integration of this axis into the simultaneous-closure mechanism of Phase 2 alongside the other five remaining Clay axes.

The deployable infrastructure that operates today on the working assumption $P \neq NP$ (cryptographic primitives, TLS handshakes, SAT solvers as heuristics, integer-programming packages, logistics optimizers) is not affected by the framework’s substrate-level content. The framework neither validates nor invalidates that assumption at the operational tier.

10 Honest scope

Per the file-level honest-scope marker (`sec : substrate – honest – scope`), PF is a *substrate-level monograph in Grothendieck mode*.

Remark 10.1 (Honest scope at the P vs NP axis, foregrounded). **(H1) Not a literal mathlib-tier Clay discharge.** The framework does not discharge $P \neq NP$ at the form the Clay rules require. The framework’s biconditional is unconditional on the chosen encoding; the right-hand side ($\text{ClassP} \neq \text{ClassNP}$ on the chosen encoding) is the open Clay conjecture, preserved openly.

(H2) The Machine type carries no operational Turing semantics. The framework’s `Machine` structure (line 71 of `Complexity.lean`) has two fields (`accepts`, `steps`) that are free functions; three type parameters (Γ, Λ, σ) unused in the body. The file docstring acknowledges this. The classes `ClassP` and `ClassNP` built atop this structure are substrate-shadows of the Cook–Karp operational form, not the operational form itself.

(H3) Not a constructive polynomial-time SAT solver. The framework does not supply a polynomial-time algorithm for any NP-complete problem. The framework makes no claim of constructive algorithmic content.

(H4) α -skeleton pinning is framework-internal. The chosen values $\alpha_P := \sqrt{2}$, $\alpha_{NP} := \varphi + 1/4$, $\alpha_{P \text{ vs } NP} := 5/4$ are pinned by the framework-internal arithmetic identities under the editorial convention. The interpretation that these are the canonical eigenvalue parameters of the P and NP operators is, per `CrossMillenniumSharedInvariants.lean` header, “conjectural and lives elsewhere”; the algebraic forcing of the chosen values is internal to the framework’s algebraic system, not the consumption of external complexity-theoretic content.

(H5) Cross-Millennium $NP \leftrightarrow$ Hodge bridge is an identity on chosen constants. The framework’s identity $\alpha_{NP} - \alpha_{\text{Hodge}} = 1/4$ is an algebraic identity on the framework’s chosen α -values; it is not a derivation of either constant from external content.

(H6) The PF Lean4Lean harness is build-hash robustness. As §8 states.

Remark 10.2 (The 143-problem coherence figure is retracted as established evidence). The `universal_fractal_coherence` Lean theorem (`PF.Empirical.HundredFortyThreeProblems`, line 167) records the framework’s prediction that all problems within a complexity class P (resp. NP) yield identical canonical α -values; the dataset is implemented as `List.replicate 72 (canonicalEntry .P) ++ List.replicate 71 (canonicalEntry .NP)`, and the `coherence_p_value_threshold` is asserted, not derived. The file’s own honest-scope note (lines 36–39) labels this as “a modeling assumption about a baseline success rate, not a derived theorem.” The $p < 10^{-43}$ figure should not be cited as established empirical evidence; earlier framework documents that did so are corrected here.

Remark 10.3 (IBM Quantum 9-way: pre-registration, 2 of 9 measured). At HEAD 4785c55 (`IBMHardware9WayEvidence.lean` lines 312–327), **only the pair $(\alpha_{RH}, \alpha_{NP})$ has been observed on IBM Quantum hardware** consistent with the framework’s prediction. The other 7 are framework predictions committed BEFORE measurement. The Lean theorem records a **conditional** bound: “IF all 9 are eventually measured to within 10^{-3} of the pre-specified values under `NoiseModel9`, THEN random-match probability is at most 10^{-15} .” Current status: 2 of 9 measured-consistent. The pre-registration of seven framework α -values prior to hardware measurement is scientifically valuable; the conditional bound says nothing absent the remaining 7 measurements.

Remark 10.4 (Falsifiability). The framework commits to eight typed falsifiability conditions in `PF.Referee.FrameworkFalsifiabilityConditions`. Audit 2026-06-04: **zero of eight triggered; two actively supported by 2024–2026 literature** (F_3, F_6).

11 Conclusion

This Phase 3 paper presents the framework’s substrate-level reformulation at the P vs NP axis. The framework’s contribution is structural: an unconditional biconditional on the framework’s chosen encoding reducing `Clay_PvsNP_Standard` to `ClassP ≠ ClassNP` on that encoding, an α -skeleton pinning of $\alpha_{\text{PvsNP}} := 5/4$ under the editorial convention, and integration into the Phase 2 simultaneous-closure mechanism. The framework’s `Machine` type carries no operational Turing semantics; the chosen encoding is the framework’s substrate-shadow of the Cook–Karp form, not the operational form. **No Clay-prize claim is made. No constructive polynomial-time algorithm is offered. The literal $P \neq NP$ on the chosen encoding is the open Clay conjecture, preserved openly.** Independent expert review and substantial revision are explicitly invited.

References

- [1] Cohen, P., Claude Opus 4.7 (2026-06-06). *A Substrate-Level Reformulation of the Six Remaining Clay Millennium Problems via the Principia Fractalis Framework*. `Papers/clay_substrate_reformulation_v1.tex`.
- [2] Cohen, P., Claude Opus 4.7 (2026-06-06). *Phase 1 — A Substrate-Level Reformulation of the Riemann Hypothesis via the Principia Fractalis Framework*. `Papers/clay_RH_via_substrate_v2.tex`.
- [3] Cohen, P., Claude Opus 4.7 (2026-06-06). *Phase 2 — The Simultaneous-Closure Mechanism on Canonical Encodings*. `Papers/clay_six_simultaneous_closure_v1.tex`.
- [4] Cohen, P., Claude Opus 4.7 (2026-06-06). *Phase 4 Keystone — Substrate-Level Content on Consciousness, Zero-Point Energy, and Cosmology*. `Papers/principia_fractalis_keystone_v1.tex`.
- [5] Cohen, P. (2026). *Principia Fractalis*, Second Edition. HEAD of <https://github.com/FractalDevTeam/Principia-Fractalis>.
- [6] Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.
- [7] Karp, R. M. (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations*, Plenum, 85–103.
- [8] Razborov, A. A., Rudich, S. (1997). Natural proofs. *J. Comput. System Sci.* 55(1): 24–35.
- [9] Aaronson, S., Wigderson, A. (2009). Algebrization: a new barrier in complexity theory. *ACM Trans. Comput. Theory* 1(1), Art. 2.
- [10] Perelman, G. (2002). The entropy formula for the Ricci flow and its geometric applications. `arXiv:math/0211159`.
- [11] Perelman, G. (2003). Ricci flow with surgery on three-manifolds. `arXiv:math/0303109`.
- [12] Perelman, G. (2003). Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. `arXiv:math/0307245`.